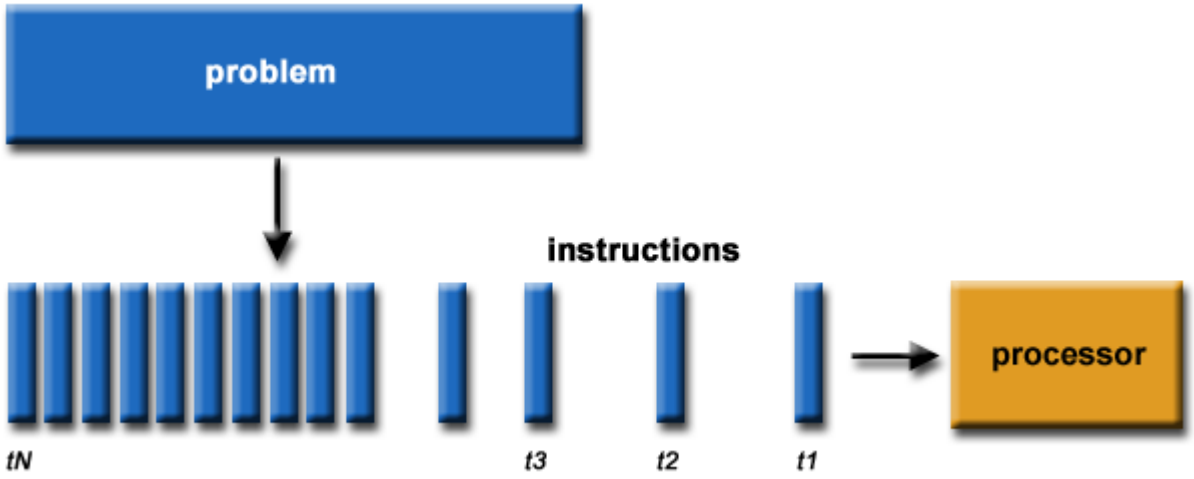
PARALLEL PROGRAMMING USING MESSAGE PASSING

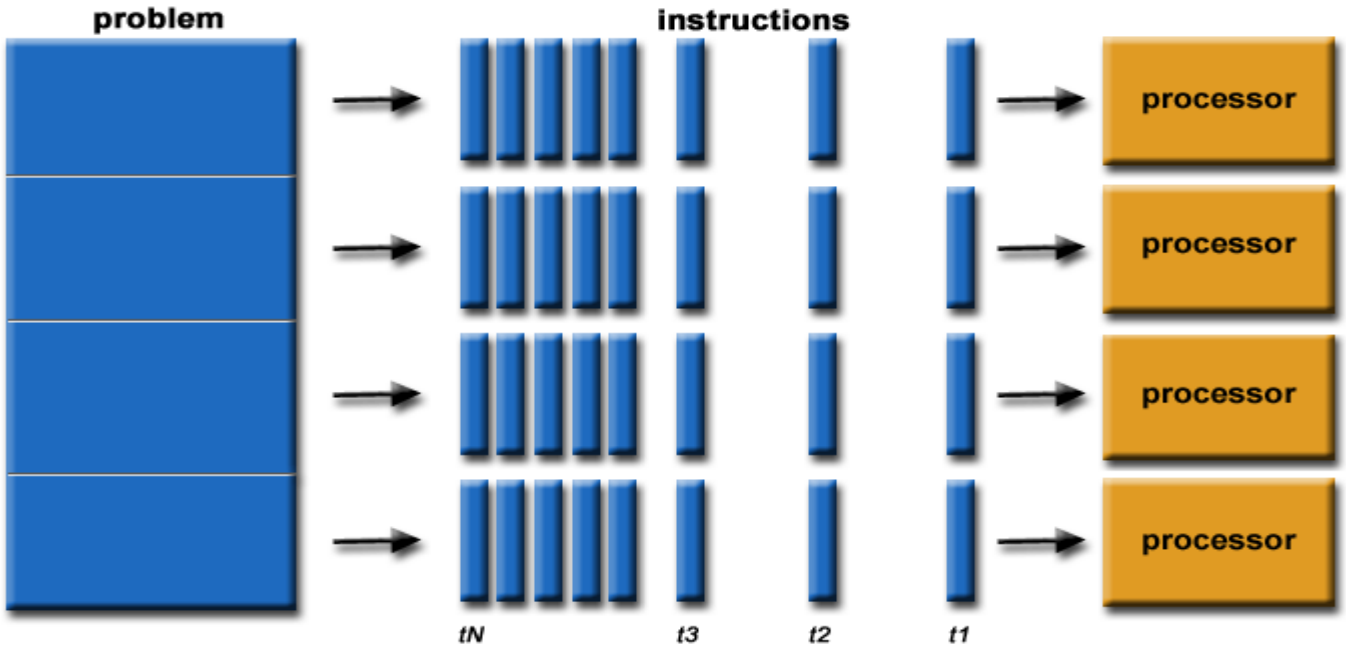
Serial Computing

- A problem is broken into a discrete series of instructions
- Instructions are executed sequentially one after another
- Executed on a single processor
- Only one instruction may execute at any moment in time



Parallel Computing

- A problem is broken into discrete parts that can be solved concurrently
- Each part is further broken down to a series of instructions
- Instructions from each part execute simultaneously on different processors
- An overall control/coordination mechanism is employed



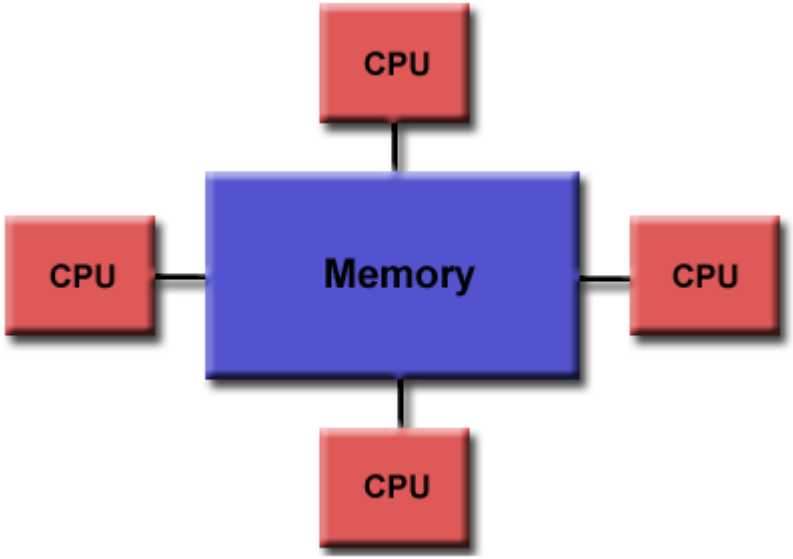
Inter-process communication models

- **Message Passing Process Communication Model**
  - Each process has its own memory
  - Communicates each other with messages

- **Shared Memory Process Communication Model**
  - processes share a common address space.
  - communicates each other through shared address space

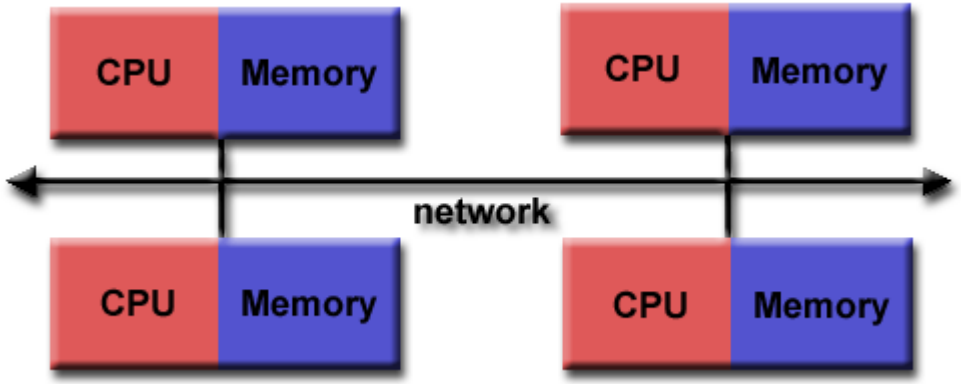
Shared Memory Architecture

- all processors to access all memory as global address space.
- Multiple processors can operate independently but share the same memory resources.
- Changes in a memory location effected by one processor are visible to all other processors.



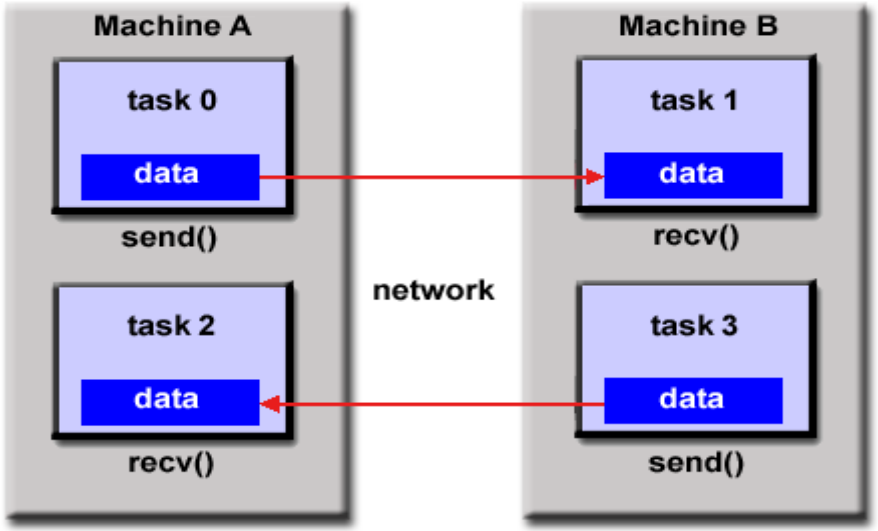
Distributed Memory Architecture

- Distributed memory systems require a communication network to connect inter-processor memory.
- Processors have their own local memory.
- Processors operate independently.



Distributed Memory / Message Passing Model

- A set of tasks that use their own local memory during computation.
- Tasks exchange data through communications by sending and receiving messages. (Movement of data from one process's address space to another's).
- Data transfer usually requires cooperative operations to be performed by each process. For example, a send operation must have a matching receive operation.



SPMD Vs MPMD

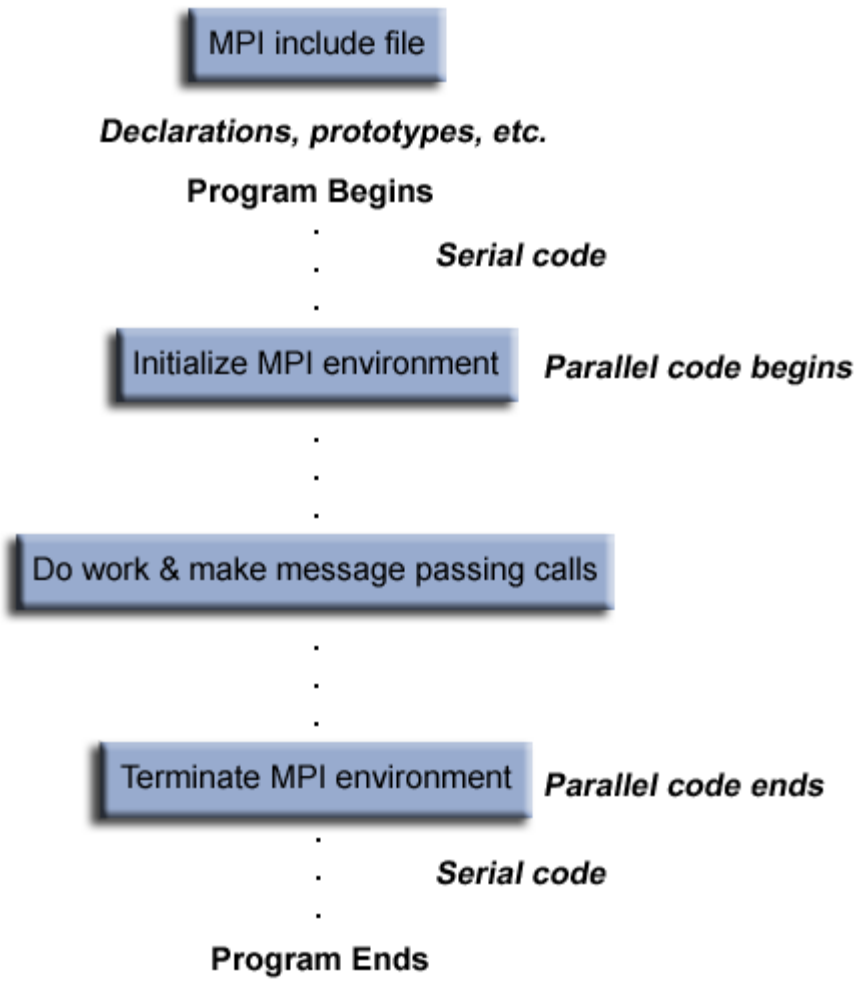
- **Single Program, Multiple Data(SPMD).**
  - Same program runs everywhere.
- **Multiple Program, Multiple Data(MPMD).**
  - Different programs run on different nodes.
  - If one program controls the others then the controlling program is called the Master and the others are called the slaves.

Message Passing Interface

- MPI is the standard for distributed memory computing.
- Specification for the developers and users of message passing libraries
  - By itself, it is NOT a library - but rather the specification of what such a library should be.
- Used for parallel computers, clusters, and heterogeneous networks as a message passing standard.

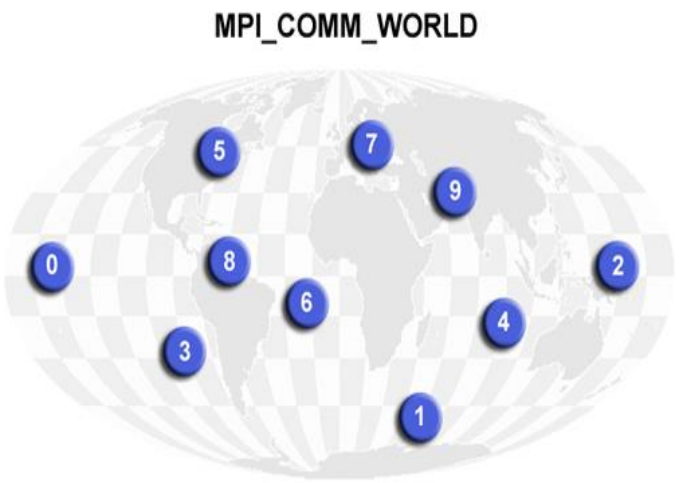
General MPI Program Structure

```
#include "stdafx.h"
#include <mpi.h> //containd the MPI function type declaractions
#include <stdio.h> // stands for "standarad input output"
int main()
{
        int np;
        int pid;
        MPI_Init(NULL,NULL) ; //Initialize the MPI execution
        enviornment
        MPI_Comm_size(MPI_COMM_WORLD,&np); //Determines
the
size of the group associated with the communicator
        MPI_COMM_WORLD is the default communicator
        MPI_Comm_rank(MPI_COMM_WORLD,&pid); //Determines
the rank of the calling process in the communicator
        printf("\n Hello world . . . \n My pid= %d \n No_of_process=
%d",pid,np);
        MPI_Finalize(); // Terminates MPI execution environment
        return 0;
}
```



General MPI Program Structure

- **Header File:**
  - Required for all programs that make MPI library calls (mpi.h).
- **Format of MPI Calls:**
  - C names are case sensitive.
  - functions with names beginning with the prefix MPI_
- **Communicators and Groups:**
  - MPI uses objects called communicators and groups to define which collection of processes may communicate with each other.
  - Use **MPI_COMM_WORLD** whenever a communicator is required - it is the predefined communicator that includes all of your MPI processes.
- **Rank:**
  - Within a communicator, every process has its own unique, integer identifier assigned by the system when the process initializes



Environment Management Routines

- **MPI_Init:**
  - Initializes the MPI execution environment.
  - must be called only once in every MPI program before any other MPI functions
- **MPI_Comm_size:**
  - Returns the total number of MPI processes in the specified communicator
- **MPI_Comm_rank:**
  - Returns the rank of the calling MPI process within the specified communicator.
  - Initially, each process will be assigned a unique integer rank between 0 and number of tasks - 1 within the communicator
- **MPI_Finalize:**
  - Terminates the MPI execution environment. This function should be the last MPI routine called in every MPI program - no other MPI routines may be called after it.

MPI Data Types

MPI data type	C data type
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_LONG_LONG_INT	long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_UNSIGNED_LONG_LONG	unsigned long long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_WCHAR	wide char
MPI_PACKED	special data type for packing
MPI_BYTE	single byte value